# ASSIGNMENT 2

## Text Pre-processing and PII Detection

| Sr. No. | Name of Student | PRN | CO Mapped |
|---|---|---|---|
| 1 | Dhawse Spandan Yashwant | 23070122085 | CO2 |
| 2 | Garvit Tyagi | 23070122094 | CO2 |

| CA3 Project Title | RedactIQ: Explainable PII Detection and Redaction with Confidence-Scored Audit Trails |
|---|---|

## Aim

To detect personally identifiable information in text by combining a rule-based regular-expression detector with a statistical named entity recogniser, to fuse and deduplicate their outputs, to assign each detection an interpretable confidence score, and to produce a redacted document together with an explainable audit trail.

The work forms the detection stage of the CA3 project RedactIQ and is evaluated using span-level precision, recall and F1.

**Learning Outcome:**

*After completing this experiment, students will be able to:*

- Implement rule-based and statistical entity detection using regular expressions and the spaCy named entity recogniser.
- Apply a fusion and confidence-scoring scheme to combine the outputs of multiple detectors into a single explainable decision.
- Evaluate and compare detection configurations using span-level precision, recall and F1.

## Theory

Personally identifiable information (PII) is any data that can identify a person, such as a name, e-mail address, telephone number or government identifier. Automated redaction is required by privacy regulations including India's DPDP Act 2023 and the EU GDPR, because manual redaction does not scale.

A rule-based detector locates PII with regular expressions. Structured identifiers obey strict surface formats, so an e-mail address, a ten-digit Indian mobile number, a PAN of the form AAAAA9999A or a twelve-digit Aadhaar number can each be matched exactly, which gives very high precision. It cannot, however, recognise names or organisations, which have no fixed form. A statistical detector solves that: Named Entity Recognition classifies token spans into predefined categories, and the pre-trained spaCy model en_core_web_sm labels PERSON and ORG entities from context and capitalisation, though it misses locale-specific identifiers and occasionally mislabels a field name such as PAN as an organisation.

Because the two detectors fail on disjoint entity classes their outputs are fused. Two detections are the same entity when their spans have an Intersection over Union above 0.5 or when one contains the other; the longer span is retained and both votes are kept. Each entity is then scored by a noisy-OR combination, conf = 1 - PROD(1 - s_i), which rises as detectors agree, so a regex vote of 0.85 with a

spaCy vote of 0.75 gives 1 - (0.15 x 0.25) = 0.9625. A further 0.05 is added when a trigger word such as contact or PAN occurs within thirty characters to the left, and the score is capped at 0.99.

The score determines the action: 0.85 and above is redacted automatically, 0.50 to 0.85 is redacted but flagged for human review, and anything lower is logged without redaction. Recording the votes, the score and the decision for every entity is what makes the system explainable rather than a black box.

The detector is evaluated at span level, with precision TP/(TP+FP), recall TP/(TP+FN) and F1 their harmonic mean. In redaction recall matters more than precision, because a missed entity is a privacy breach whereas an over-redacted word is only a loss of readability; the leakage rate, one minus recall, is therefore reported as well.

## Algorithm

1. Import the re module and load the pre-trained spaCy model en_core_web_sm.
2. Define the regular-expression patterns for EMAIL, PHONE, AADHAAR and PAN, each with its base confidence score, and define the list of trigger words.
3. Detector 1: search the text with every pattern and record the matched string, its character span, its label and its base score.
4. Detector 2: run the spaCy pipeline and record every PERSON and ORG entity, discarding entities whose text is itself a field name such as PAN or Aadhaar.
5. Fuse the two lists: sort the detections by starting position and merge any pair whose spans have an IoU above 0.5 or where one contains the other, keeping the longer span and both detector votes.
6. Compute the confidence of each entity as one minus the product of the complements of its votes, add 0.05 if a trigger word occurs to its left, and cap the result at 0.99.
7. Assign a decision band: AUTO_REDACT above 0.85, REVIEW between 0.50 and 0.85, and LOG_ONLY below 0.50.
8. Replace every non-LOG_ONLY entity from right to left with a placeholder of the form [LABEL_REDACTED], so that the earlier character offsets remain valid.
9. Print the redacted text and the audit trail listing the span, type, votes, confidence and decision of each entity.
10. Evaluate the configurations regex only, spaCy only and fusion of both on a labelled test set, and report TP, FP, FN, precision, recall, F1 and leakage rate.

## Program

```python
import re
import textwrap
import spacy

nlp = spacy.load("en_core_web_sm")

# Regex patterns for structured PII, with the base score of each
PATTERNS = {
    "EMAIL":   (r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}", 0.85),
    "PHONE":   (r"(?:\+91[-\s]?|0)?[6-9]\d{9}\b",                  0.85),
    "AADHAAR": (r"\b[2-9]\d{3}\s?\d{4}\s?\d{4}\b",                 0.95),
    "PAN":     (r"\b[A-Z]{5}\d{4}[A-Z]\b",                         0.95),
}
SPACY_MAP = {"PERSON": "PERSON", "ORG": "ORG"}
BLOCK = {"pan", "aadhaar", "dob"}          # field names, not values
TRIGGERS = ("email", "contact", "phone", "mobile", "pan", "aadhaar")

def regex_detect(text):
    """Detector 1 : rule based."""
    return [{"text": m.group(), "start": m.start(), "end": m.end(),
             "label": lab, "src": ["regex"], "sc": [base]}
            for lab, (pat, base) in PATTERNS.items()
            for m in re.finditer(pat, text)]

def spacy_detect(text):
    """Detector 2 : statistical NER."""
    return [{"text": e.text, "start": e.start_char, "end": e.end_char,
             "label": SPACY_MAP[e.label_], "src": ["spacy"], "sc": [0.75]}
            for e in nlp(text).ents
            if e.label_ in SPACY_MAP and e.text.lower() not in BLOCK]
```

```python
def iou(a, b):
    """Intersection over union of two character spans."""
    inter = max(0, min(a["end"], b["end"]) - max(a["start"], b["start"]))
    union = max(a["end"], b["end"]) - min(a["start"], b["start"])
    return inter / union if union else 0.0

def same(a, b):
    """Same entity if the spans overlap well or one contains the other."""
    return (iou(a, b) > 0.5
            or (a["start"] <= b["start"] and b["end"] <= a["end"])
            or (b["start"] <= a["start"] and a["end"] <= b["end"]))

def fuse(ents):
    """Merge duplicate detections, keeping the longer span and both votes."""
    merged = []
    for e in sorted(ents, key=lambda x: (x["start"], -(x["end"] - x["start"]))):
        for m in merged:
            if same(m, e):
                m["src"] += e["src"]
                m["sc"] += e["sc"]
                break
        else:
            merged.append(dict(e))
    return merged
```

```python
def confidence(e, text):
    """Noisy-OR over the detector votes, plus a context boost."""
    p = 1.0
    for s in e["sc"]:
        p *= (1 - s)
    conf = 1 - p
    if any(w in text[max(0, e["start"] - 30):e["start"]].lower() for w in TRIGGERS):
        conf += 0.05
    return min(conf, 0.99)

def band(c):
    return "AUTO_REDACT" if c >= 0.85 else ("REVIEW" if c >= 0.5 else "LOG_ONLY")

def analyse(text, use_regex=True, use_spacy=True):
    ents = (regex_detect(text) if use_regex else []) + \
           (spacy_detect(text) if use_spacy else [])
    ents = sorted(fuse(ents), key=lambda x: x["start"])
    for e in ents:
        e["conf"] = confidence(e, text)
        e["band"] = band(e["conf"])
    return ents

def redact(text, ents):
    for e in sorted(ents, key=lambda x: -x["start"]):    # replace right to left
        if e["band"] != "LOG_ONLY":
            text = text[:e["start"]] + f"[{e['label']}_REDACTED]" + text[e["end"]:]
    return text
```

```python
DEMO = ("Please contact Rohan Mehta at rohan.mehta@sitpune.edu.in or on "
        "+91-9876543210. His PAN is ABCDE1234F and Aadhaar 4321 8765 2109. "
        "The file was forwarded to Priya Sharma at Infosys Limited.")

ents = analyse(DEMO)
print("REDACTED OUTPUT")
print(textwrap.fill(redact(DEMO, ents), width=86,
                    initial_indent="  ", subsequent_indent="  "))

print("\nEXPLAINABLE AUDIT TRAIL")
print(f"  {'Entity':<28}{'Span':<11}{'Type':<9}{'Votes':<14}{'Conf':<7}Decision")
for e in ents:
    votes = "+".join(sorted(set(e["src"])))
    print(f"  {e['text'][:26]:<28}{str(e['start'])+'-'+str(e['end']):<11}"
          f"{e['label']:<9}{votes:<14}{e['conf']:<7.3f}{e['band']}")
```

```python
# ---------------- Evaluation on a small labelled test set ----------------
GOLD = [
 ("Employee Kabir Rao of Wipro Technologies has PAN ZXCVB9876K.",
  [("Kabir Rao", "PERSON"), ("Wipro Technologies", "ORG"), ("ZXCVB9876K", "PAN")]),
 ("Mail Anita Desai at anita.desai@gmail.com or call 9812345670.",
  [("Anita Desai", "PERSON"), ("anita.desai@gmail.com", "EMAIL"),
   ("9812345670", "PHONE")]),
 ("Aadhaar 5678 1234 9012 belongs to Farhan Qureshi.",
  [("5678 1234 9012", "AADHAAR"), ("Farhan Qureshi", "PERSON")]),
 ("The compliance review meeting is scheduled next week.", []),
]
```

```python
def evaluate(use_regex, use_spacy):
    tp = fp = fn = 0
    for text, gold in GOLD:
        pred = [(e["text"], e["label"]) for e in analyse(text, use_regex, use_spacy)]
        left = list(gold)
        for p in pred:
            if p in left:
                left.remove(p)
                tp += 1
            else:
                fp += 1
        fn += len(left)
    P = tp / (tp + fp) if tp + fp else 0.0
    R = tp / (tp + fn) if tp + fn else 0.0
    F = 2 * P * R / (P + R) if P + R else 0.0
    return tp, fp, fn, P, R, F

print("\nEVALUATION ON THE LABELLED TEST SET")
print(f"  {'Configuration':<26}{'TP':<5}{'FP':<5}{'FN':<5}"
      f"{'Prec':<8}{'Recall':<9}{'F1':<8}Leakage")
for name, rg, sp in [("C1  regex only", True, False),
                     ("C2  spaCy NER only", False, True),
                     ("C3  fusion of both", True, True)]:
    tp, fp, fn, P, R, F = evaluate(rg, sp)
    print(f"  {name:<26}{tp:<5}{fp:<5}{fn:<5}{P:<8.3f}{R:<9.3f}{F:<8.3f}{1-R:.3f}")
```

## Output

```
REDACTED OUTPUT
  Please contact [PERSON_REDACTED] at [EMAIL_REDACTED] or on [PHONE_REDACTED]. His PAN
  is [PAN_REDACTED] and Aadhaar [AADHAAR_REDACTED]. The file was forwarded to
  [PERSON_REDACTED] at [ORG_REDACTED].
```

```
EXPLAINABLE AUDIT TRAIL
  Entity                    Span      Type     Votes         Conf   Decision
  Rohan Mehta               15-26     PERSON   spacy         0.800  REVIEW
  rohan.mehta@sitpune.edu.in 30-56    EMAIL    regex         0.900  AUTO_REDACT
  +91-9876543210            63-77     PHONE    regex         0.850  AUTO_REDACT
  ABCDE1234F                90-100    PAN      regex+spacy   0.990  AUTO_REDACT
  4321 8765 2109            113-127   AADHAAR  regex         0.990  AUTO_REDACT
  Priya Sharma              155-167   PERSON   spacy         0.750  REVIEW
  Infosys Limited           171-186   ORG      spacy         0.750  REVIEW
```

```
EVALUATION ON THE LABELLED TEST SET
  Configuration             TP   FP   FN   Prec    Recall   F1       Leakage
  C1  regex only            4    0    4    1.000   0.500    0.667    0.500
  C2  spaCy NER only        4    0    4    1.000   0.500    0.667    0.500
  C3  fusion of both        8    0    0    1.000   1.000    1.000    0.000
```

### Explanation of the Output

The redacted output shows every identifier replaced by a typed placeholder while the surrounding sentence structure is preserved, so the document remains readable and its meaning is retained.

In the audit trail the e-mail address, telephone number and Aadhaar number are found by the regular expressions alone, and the two person names and the organisation by spaCy alone, which confirms that the two detectors cover disjoint classes. The PAN number is the only entity found by both, and its confidence rises to 0.990 through the noisy-OR combination. The effect of the context boost is visible on Rohan Mehta, which scores 0.800 rather than 0.750 because the trigger word contact precedes it; the two entities that fall in the 0.50 to 0.85 band are flagged for review rather than redacted silently.

The evaluation confirms the design quantitatively. Each detector alone reaches a recall of only 0.500 and so leaks half the PII, while their fusion recovers every entity with no false positives. The test set here is deliberately small, so the perfect fusion score should be read as evidence that the two detectors are complementary rather than as a general accuracy estimate.

## Conclusion

In this experiment a two-detector PII detection pipeline was implemented and evaluated. Regular expressions were used to locate structured identifiers with high precision, and the pre-trained spaCy named entity recogniser was used to locate person and organisation names that have no fixed surface form. The two outputs were fused by an Intersection over Union and containment rule, each surviving entity was scored by a noisy-OR combination of its detector votes together with a context boost, and the score was mapped to one of three decision bands. The results show that either detector used alone leaks half of the PII in the test set, whereas their fusion recovers all of it, which demonstrates empirically that the two methods fail on disjoint entity classes and are therefore complementary. Recording the votes, the confidence and the reason for every decision produces an audit trail that a compliance reviewer can inspect, which is the property that distinguishes RedactIQ from a conventional redaction tool. The detections produced here are used as the input to the feature engineering carried out in the next assignment.